

DATA STRUCTURES AND ALGORITHMS MADE EASY

Data Structures and Algorithmic Puzzles

5TH
EDITION



Narasimha Karumanchi, M.Tech, IIT Bombay
Founder, CareerMonk.com

Data Structures And Algorithms Made Easy

-To All My Readers

**By
Narasimha Karumanchi**

 **Concepts**  **Problems**  **Interview Questions**

Copyright© 2017 by CareerMonk.com

All rights reserved.

Designed by *Narasimha Karumanchi*

Copyright© 2017 CareerMonk Publications. All rights reserved.

All rights reserved. No part of this book may be reproduced in any form or by any electronic or mechanical means, including information storage and retrieval systems, without written permission from the publisher or author.

Acknowledgements

Mother and Father, it is impossible to thank you adequately for everything you have done, from loving me unconditionally to raising me in a stable household, where your persistent efforts and traditional values taught your children to celebrate and embrace life. I could not have asked for better parents or role-models. You showed me that anything is possible with faith, hard work and determination.

This book would not have been possible without the help of many people. I would like to express my gratitude to all of the people who provided support, talked things over, read, wrote, offered comments, allowed me to quote their remarks and assisted in the editing, proofreading and design. In particular, I would like to thank the following individuals:

- *Mohan Mullapudi*, IIT Bombay, Architect, dataRPM Pvt. Ltd.
- *Navin Kumar Jaiswal*, Senior Consultant, Juniper Networks Inc.
- *A. Vamshi Krishna*, IIT Kanpur, Mentor Graphics Inc.
- *Cathy Reed*, BA, MA, Copy Editor

–*Narasimha Karumanchi*
M-Tech, IIT Bombay
Founder, CareerMonk.com

Preface

Dear Reader,

Please hold on! I know many people typically do not read the Preface of a book. But I strongly recommend that you read this particular Preface.

It is not the main objective of this book to present you with the theorems and proofs on *data structures* and *algorithms*. I have followed a pattern of improving the problem solutions with different complexities (for each problem, you will find multiple solutions with different, and reduced, complexities). Basically, it's an enumeration of possible solutions. With this approach, even if you get a new question, it will show you a way to *think* about the possible solutions. You will find this book useful for interview preparation, competitive exams preparation, and campus interview preparations.

As a *job seeker*, if you read the complete book, I am sure you will be able to challenge the interviewers. If you read it as an *instructor*, it will help you to deliver lectures with an approach that is easy to follow, and as a result your students will appreciate the fact that they have opted for Computer Science / Information Technology as their degree.

This book is also useful for *Engineering degree students* and *Masters degree students* during their academic preparations. In all the chapters you will see that there is more emphasis on problems and their analysis rather than on theory. In each chapter, you will first read about the basic required theory, which is then followed by a section on problem sets. In total, there are approximately 700 algorithmic problems, all with solutions.

If you read the book as a *student* preparing for competitive exams for Computer Science / Information Technology, the content covers *all the required topics* in full detail. While writing this book, my main focus was to help students who are preparing for these exams.

In all the chapters you will see more emphasis on problems and analysis rather than on theory. In each chapter, you will first see the basic required theory followed by various problems.

For many problems, *multiple* solutions are provided with different levels of complexity. We start with the *brute force* solution and slowly move toward the *best solution* possible for that problem. For each problem, we endeavor to understand how much time the algorithm takes and how much memory the algorithm uses.

It is recommended that the reader does at least one *complete* reading of this book to gain a full understanding of all the topics that are covered. Then, in subsequent readings you can skip directly to any chapter to refer to a specific topic. Even though many readings have been done for the purpose of correcting errors, there could still be some minor typos in the book. If any are found, they will be updated at www.CareerMonk.com. You can monitor this site for any corrections and also for new problems and solutions. Also, please provide your valuable suggestions at: Info@CareerMonk.com.

I wish you all the best and I am confident that you will find this book useful.

–Narasimha Karumanchi
M-Tech, IIT Bombay
Founder, CareerMonk.com

Other Books by Narasimha Karumanchi



IT Interview Questions



Data Structures and Algorithms for GATE



Data Structures and Algorithms Made Easy in Java



Coding Interview Questions



Peeling Design Patterns



Elements of Computer Networking



Data Structures and Algorithmic Thinking with Python

Table of Contents

1. Introduction

- 1.1 Variables
- 1.2 Data Types
- 1.3 Data Structures
- 1.4 Abstract Data Types (ADTs)
- 1.5 What is an Algorithm?
- 1.6 Why the Analysis of Algorithms?
- 1.7 Goal of the Analysis of Algorithms
- 1.8 What is Running Time Analysis?
- 1.9 How to Compare Algorithms
- 1.10 What is Rate of Growth?
- 1.11 Commonly Used Rates of Growth
- 1.12 Types of Analysis
- 1.13 Asymptotic Notation
- 1.14 Big-O Notation [Upper Bounding Function]
- 1.15 Omega- Ω Notation [Lower Bounding Function]
- 1.16 Theta- Θ Notation [Order Function]
- 1.17 Important Notes
- 1.18 Why is it called Asymptotic Analysis?
- 1.19 Guidelines for Asymptotic Analysis
- 1.20 Simplifying properties of asymptotic notations
- 1.21 Commonly used Logarithms and Summations
- 1.22 Master Theorem for Divide and Conquer Recurrences
- 1.23 Divide and Conquer Master Theorem: Problems & Solutions
- 1.24 Master Theorem for Subtract and Conquer Recurrences
- 1.25 Variant of Subtraction and Conquer Master Theorem
- 1.26 Method of Guessing and Confirming

1.27 Amortized Analysis

1.28 Algorithms Analysis: Problems & Solutions

2. Recursion and Backtracking

2.1 Introduction

2.2 What is Recursion?

2.3 Why Recursion?

2.4 Format of a Recursive Function

2.5 Recursion and Memory (Visualization)

2.6 Recursion versus Iteration

2.7 Notes on Recursion

2.8 Example Algorithms of Recursion

2.9 Recursion: Problems & Solutions

2.10 What is Backtracking?

2.11 Example Algorithms of Backtracking

2.12 Backtracking: Problems & Solutions

3. Linked Lists

3.1 What is a Linked List?

3.2 Linked Lists ADT

3.3 Why Linked Lists?

3.4 Arrays Overview

3.5 Comparison of Linked Lists with Arrays & Dynamic Arrays

3.6 Singly Linked Lists

3.7 Doubly Linked Lists

3.8 Circular Linked Lists

3.9 A Memory-efficient Doubly Linked List

3.10 Unrolled Linked Lists

3.11 Skip Lists

3.12 Linked Lists: Problems & Solutions

4. Stacks

4.1 What is a Stack?

4.2 How Stacks are used

4.3 Stack ADT

- 4.4 Applications
- 4.5 Implementation
- 4.6 Comparison of Implementations
- 4.7 Stacks: Problems & Solutions

5. Queues

- 5.1 What is a Queue?
- 5.2 How are Queues Used?
- 5.3 Queue ADT
- 5.4 Exceptions
- 5.5 Applications
- 5.6 Implementation
- 5.7 Queues: Problems & Solutions

6. Trees

- 6.1 What is a Tree?
- 6.2 Glossary
- 6.3 Binary Trees
- 6.4 Types of Binary Trees
- 6.5 Properties of Binary Trees
- 6.6 Binary Tree Traversals
- 6.7 Generic Trees (N -ary Trees)
- 6.8 Threaded Binary Tree Traversals (Stack or Queue-less Traversals)
- 6.9 Expression Trees
- 6.10 XOR Trees
- 6.11 Binary Search Trees (BSTs)
- 6.12 Balanced Binary Search Trees
- 6.13 AVL (Adelson-Velskii and Landis) Trees
- 6.14 Other Variations on Trees

7. Priority Queues and Heaps

- 7.1 What is a Priority Queue?
- 7.2 Priority Queue ADT
- 7.3 Priority Queue Applications
- 7.4 Priority Queue Implementations

- 7.5 Heaps and Binary Heaps
- 7.6 Binary Heaps
- 7.7 Heapsort
- 7.8 Priority Queues [Heaps]: Problems & Solutions

8. Disjoint Sets ADT

- 8.1 Introduction
- 8.2 Equivalence Relations and Equivalence Classes
- 8.3 Disjoint Sets ADT
- 8.4 Applications
- 8.5 Tradeoffs in Implementing Disjoint Sets ADT
- 8.8 Fast UNION Implementation (Slow FIND)
- 8.9 Fast UNION Implementations (Quick FIND)
- 8.10 Summary
- 8.11 Disjoint Sets: Problems & Solutions

9. Graph Algorithms

- 9.1 Introduction
- 9.2 Glossary
- 9.3 Applications of Graphs
- 9.4 Graph Representation
- 9.5 Graph Traversals
- 9.6 Topological Sort
- 9.7 Shortest Path Algorithms
- 9.8 Minimal Spanning Tree
- 9.9 Graph Algorithms: Problems & Solutions

10. Sorting

- 10.1 What is Sorting?
- 10.2 Why is Sorting Necessary?
- 10.3 Classification of Sorting Algorithms
- 10.4 Other Classifications
- 10.5 Bubble Sort
- 10.6 Selection Sort
- 10.7 Insertion Sort

- 10.8 Shell Sort
- 10.9 Merge Sort
- 10.10 Heap Sort
- 10.11 Quick Sort
- 10.12 Tree Sort
- 10.13 Comparison of Sorting Algorithms
- 10.14 Linear Sorting Algorithms
- 10.15 Counting Sort
- 10.16 Bucket Sort (or Bin Sort)
- 10.17 Radix Sort
- 10.18 Topological Sort
- 10.19 External Sorting
- 10.20 Sorting: Problems & Solutions

11. Searching

- 11.1 What is Searching?
- 11.2 Why do we need Searching?
- 11.3 Types of Searching
- 11.4 Unordered Linear Search
- 11.5 Sorted/Ordered Linear Search
- 11.6 Binary Search
- 11.7 Interpolation Search
- 11.8 Comparing Basic Searching Algorithms
- 11.9 Symbol Tables and Hashing
- 11.10 String Searching Algorithms
- 11.11 Searching: Problems & Solutions

12. Selection Algorithms [Medians]

- 12.1 What are Selection Algorithms?
- 12.2 Selection by Sorting
- 12.3 Partition-based Selection Algorithm
- 12.4 Linear Selection Algorithm - Median of Medians Algorithm
- 12.5 Finding the K Smallest Elements in Sorted Order
- 12.6 Selection Algorithms: Problems & Solutions

13. Symbol Tables

- 13.1 Introduction
- 13.2 What are Symbol Tables?
- 13.3 Symbol Table Implementations
- 13.4 Comparison Table of Symbols for Implementations

14. Hashing

- 14.1 What is Hashing?
- 14.2 Why Hashing?
- 14.3 HashTable ADT
- 14.4 Understanding Hashing
- 14.5 Components of Hashing
- 14.6 Hash Table
- 14.7 Hash Function
- 14.8 Load Factor
- 14.9 Collisions
- 14.10 Collision Resolution Techniques
- 14.11 Separate Chaining
- 14.12 Open Addressing
- 14.13 Comparison of Collision Resolution Techniques
- 14.14 How Hashing Gets $O(1)$ Complexity?
- 14.15 Hashing Techniques
- 14.16 Problems for which Hash Tables are not suitable
- 14.17 Bloom Filters
- 14.18 Hashing: Problems & Solutions

15. String Algorithms

- 15.1 Introduction
- 15.2 String Matching Algorithms
- 15.3 Brute Force Method
- 15.4 Rabin-Karp String Matching Algorithm
- 15.5 String Matching with Finite Automata
- 15.6 KMP Algorithm
- 15.7 Boyer-Moore Algorithm

- 15.8 Data Structures for Storing Strings
- 15.9 Hash Tables for Strings
- 15.10 Binary Search Trees for Strings
- 15.11 Tries
- 15.12 Ternary Search Trees
- 15.13 Comparing BSTs, Tries and TSTs
- 15.14 Suffix Trees
- 15.15 String Algorithms: Problems & Solutions

16. Algorithms Design Techniques

- 16.1 Introduction
- 16.2 Classification
- 16.3 Classification by Implementation Method
- 16.4 Classification by Design Method
- 16.5 Other Classifications

17. Greedy Algorithms

- 17.1 Introduction
- 17.2 Greedy Strategy
- 17.3 Elements of Greedy Algorithms
- 17.4 Does Greedy Always Work?
- 17.5 Advantages and Disadvantages of Greedy Method
- 17.6 Greedy Applications
- 17.7 Understanding Greedy Technique
- 17.8 Greedy Algorithms: Problems & Solutions

18. Divide and Conquer Algorithms

- 18.1 Introduction
- 18.2 What is the Divide and Conquer Strategy?
- 18.3 Does Divide and Conquer Always Work?
- 18.4 Divide and Conquer Visualization
- 18.5 Understanding Divide and Conquer
- 18.6 Advantages of Divide and Conquer
- 18.7 Disadvantages of Divide and Conquer
- 18.8 Master Theorem

18.9 Divide and Conquer Applications

18.10 Divide and Conquer: Problems & Solutions

19. Dynamic Programming

19.1 Introduction

19.2 What is Dynamic Programming Strategy?

19.3 Properties of Dynamic Programming Strategy

19.4 Can Dynamic Programming Solve All Problems?

19.5 Dynamic Programming Approaches

19.6 Examples of Dynamic Programming Algorithms

19.7 Understanding Dynamic Programming

19.8 Longest Common Subsequence

19.9 Dynamic Programming: Problems & Solutions

20. Complexity Classes

20.1 Introduction

20.2 Polynomial/Exponential Time

20.3 What is a Decision Problem?

20.4 Decision Procedure

20.5 What is a Complexity Class?

20.6 Types of Complexity Classes

20.7 Reductions

20.8 Complexity Classes: Problems & Solutions

21. Miscellaneous Concepts

21.1 Introduction

21.2 Hacks on Bit-wise Programming

21.3 Other Programming Questions

References

INTRODUCTION

CHAPTER

1



The objective of this chapter is to explain the importance of the analysis of algorithms, their notations, relationships and solving as many problems as possible. Let us first focus on understanding the basic elements of algorithms, the importance of algorithm analysis, and then slowly move toward the other topics as mentioned above. After completing this chapter, you should be able to find the complexity of any given algorithm (especially recursive functions).

1.1 Variables

Before going to the definition of variables, let us relate them to old mathematical equations. All of us have solved many mathematical equations since childhood. As an example, consider the below equation:

$$x^2 + 2y - 2 = 1$$

We don't have to worry about the use of this equation. The important thing that we need to understand is that the equation has names (x and y), which hold values (data). That means the *names* (x and y) are placeholders for representing data. Similarly, in computer science programming we need something for holding data, and *variables* is the way to do that.

1.2 Data Types

In the above-mentioned equation, the variables x and y can take any values such as integral numbers (10, 20), real numbers (0.23, 5.5), or just 0 and 1. To solve the equation, we need to relate them to the kind of values they can take, and *data type* is the name used in computer science programming for this purpose. A *data type* in a programming language is a set of data with predefined values. Examples of data types are: integer, floating point, unit number, character, string, etc.

Computer memory is all filled with zeros and ones. If we have a problem and we want to code it, it's very difficult to provide the solution in terms of zeros and ones. To help users, programming languages and compilers provide us with data types. For example, *integer* takes 2 bytes (actual value depends on compiler), *float* takes 4 bytes, etc. This says that in memory we are combining 2 bytes (16 bits) and calling it an *integer*. Similarly, combining 4 bytes (32 bits) and calling it a *float*. A data type reduces the coding effort. At the top level, there are two types of data types:

- System-defined data types (also called *Primitive* data types)
- User-defined data types

System-defined data types (Primitive data types)

Data types that are defined by system are called *primitive* data types. The primitive data types provided by many programming languages are: int, float, char, double, bool, etc. The number of bits allocated for each primitive data type depends on the programming languages, the compiler and the operating system. For the same primitive data type, different languages may use different sizes. Depending on the size of the data types, the total available values (domain) will also change.

For example, "*int*" may take 2 bytes or 4 bytes. If it takes 2 bytes (16 bits), then the total possible values are minus 32,768 to plus 32,767 (-2^{15} to $2^{15}-1$). If it takes 4 bytes (32 bits), then the possible values are between -2,147,483,648 and +2,147,483,647 (-2^{31} to $2^{31}-1$). The same is the case with other data types.

User defined data types

If the system-defined data types are not enough, then most programming languages allow the users

to define their own data types, called *user – defined data types*. Good examples of user defined data types are: structures in *C/C + +* and classes in *Java*. For example, in the snippet below, we are combining many system-defined data types and calling the user defined data type by the name “*newType*”. This gives more flexibility and comfort in dealing with computer memory.

```
struct newType {  
    int data1;  
    float data 2;  
    ...  
    char data;  
};
```

1.3 Data Structures

Based on the discussion above, once we have data in variables, we need some mechanism for manipulating that data to solve problems. *Data structure* is a particular way of storing and organizing data in a computer so that it can be used efficiently. A *data structure* is a special format for organizing and storing data. General data structure types include arrays, files, linked lists, stacks, queues, trees, graphs and so on.

Depending on the organization of the elements, data structures are classified into two types:

- 1) *Linear data structures*: Elements are accessed in a sequential order but it is not compulsory to store all elements sequentially. *Examples*: Linked Lists, Stacks and Queues.
- 2) *Non – linear data structures*: Elements of this data structure are stored/accessed in a non-linear order. *Examples*: Trees and graphs.

1.4 Abstract Data Types (ADTs)

Before defining abstract data types, let us consider the different view of system-defined data types. We all know that, by default, all primitive data types (int, float, etc.) support basic operations such as addition and subtraction. The system provides the implementations for the primitive data types. For user-defined data types we also need to define operations. The implementation for these operations can be done when we want to actually use them. That means, in general, user defined data types are defined along with their operations.

To simplify the process of solving problems, we combine the data structures with their operations and we call this *Abstract Data Types* (ADTs). An ADT consists of *two parts*:

1. Declaration of data

2. Declaration of operations

Commonly used ADTs *include*: Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Dictionaries, Disjoint Sets (Union and Find), Hash Tables, Graphs, and many others. For example, stack uses LIFO (Last-In-First-Out) mechanism while storing the data in data structures. The last element inserted into the stack is the first element that gets deleted. Common operations of it are: creating the stack, pushing an element onto the stack, popping an element from stack, finding the current top of the stack, finding number of elements in the stack, etc.

While defining the ADTs do not worry about the implementation details. They come into the picture only when we want to use them. Different kinds of ADTs are suited to different kinds of applications, and some are highly specialized to specific tasks. By the end of this book, we will go through many of them and you will be in a position to relate the data structures to the kind of problems they solve.

1.5 What is an Algorithm?

Let us consider the problem of preparing an *omelette*. To prepare an omelette, we follow the steps given below:

- 1) Get the frying pan.
- 2) Get the oil.
 - a. Do we have oil?
 - i. If yes, put it in the pan.
 - ii. If no, do we want to buy oil?
 1. If yes, then go out and buy.
 2. If no, we can terminate.
- 3) Turn on the stove, etc...

What we are doing is, for a given problem (preparing an omelette), we are providing a step-by-step procedure for solving it. The formal definition of an algorithm can be stated as:

An algorithm is the step-by-step unambiguous instructions to solve a given problem.

In the traditional study of algorithms, there are two main criteria for judging the merits of algorithms: correctness (does the algorithm give solution to the problem in a finite number of steps?) and efficiency (how much resources (in terms of memory and time) does it take to execute the).

Note: We do not have to prove each step of the algorithm.

1.6 Why the Analysis of Algorithms?

To go from city “A” to city “B”, there can be many ways of accomplishing this: by flight, by bus, by train and also by bicycle. Depending on the availability and convenience, we choose the one that suits us. Similarly, in computer science, multiple algorithms are available for solving the same problem (for example, a sorting problem has many algorithms, like insertion sort, selection sort, quick sort and many more). Algorithm analysis helps us to determine which algorithm is most efficient in terms of time and space consumed.

1.7 Goal of the Analysis of Algorithms

The goal of the *analysis of algorithms* is to compare algorithms (or solutions) mainly in terms of running time but also in terms of other factors (e.g., memory, developer effort, etc.)

1.8 What is Running Time Analysis?

It is the process of determining how processing time increases as the size of the problem (input size) increases. Input size is the number of elements in the input, and depending on the problem type, the input may be of different types. The following are the common types of inputs.

- Size of an array
- Polynomial degree
- Number of elements in a matrix
- Number of bits in the binary representation of the input
- Vertices and edges in a graph.

1.9 How to Compare Algorithms

To compare algorithms, let us define a *few objective measures*:

Execution times? *Not a good measure* as execution times are specific to a particular computer.

Number of statements executed? *Not a good measure*, since the number of statements varies with the programming language as well as the style of the individual programmer.

Ideal solution? Let us assume that we express the running time of a given algorithm as a function of the input size n (i.e., $f(n)$) and compare these different functions corresponding to running times. This kind of comparison is independent of machine time, programming style, etc.

1.10 What is Rate of Growth?

The rate at which the running time increases as a function of input is called *rate of growth*. Let us

assume that you go to a shop to buy a car and a bicycle. If your friend sees you there and asks what you are buying, then in general you say *buying a car*. This is because the cost of the car is high compared to the cost of the bicycle (approximating the cost of the bicycle to the cost of the car).

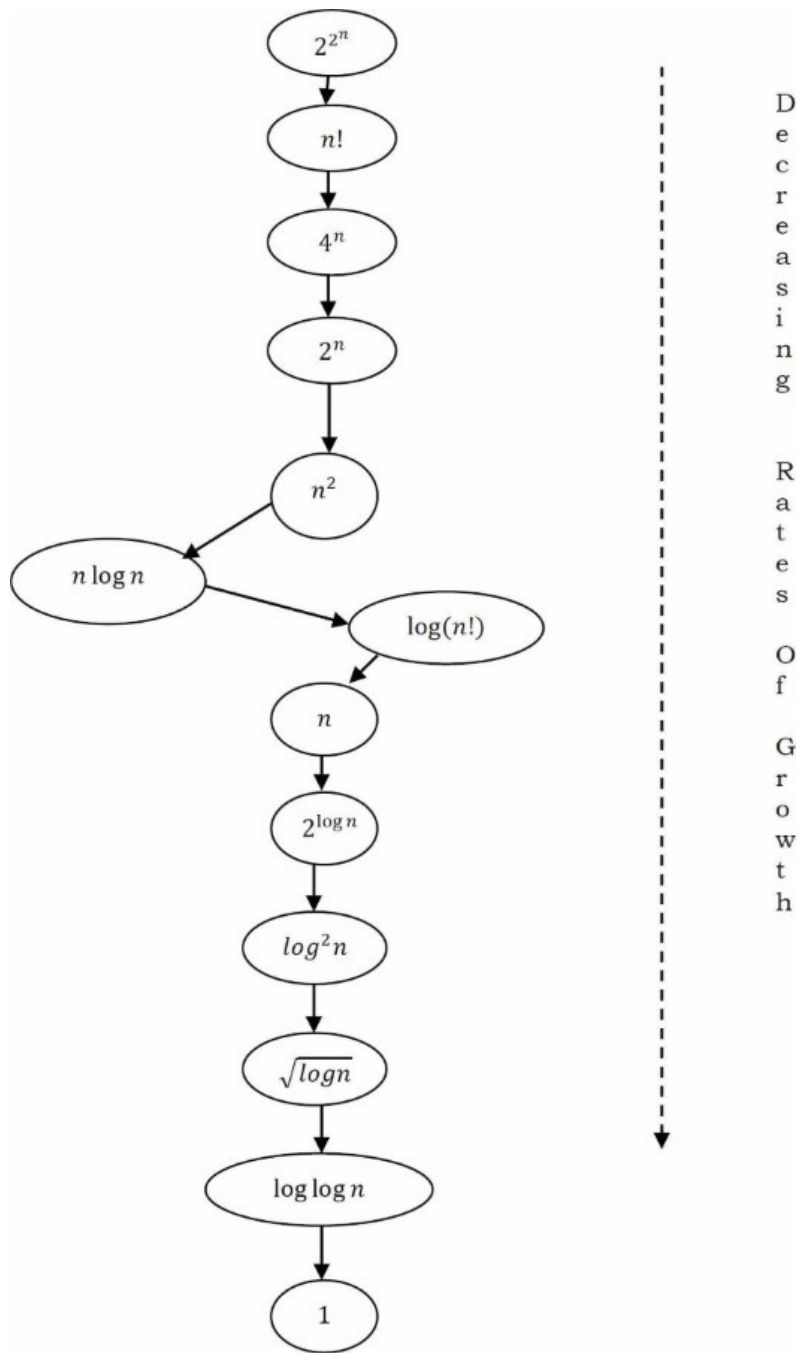
$$\begin{aligned} \text{Total Cost} &= \text{cost_of_car} + \text{cost_of_bicycle} \\ \text{Total Cost} &\approx \text{cost_of_car} \text{ (approximation)} \end{aligned}$$

For the above-mentioned example, we can represent the cost of the car and the cost of the bicycle in terms of function, and for a given function ignore the low order terms that are relatively insignificant (for large value of input size, n). As an example, in the case below, n^4 , $2n^2$, $100n$ and 500 are the individual costs of some function and approximate to n^4 since n^4 is the highest rate of growth.

$$n^4 + 2n^2 + 100n + 500 \approx n^4$$

1.11 Commonly Used Rates of Growth

The diagram below shows the relationship between different rates of growth.



Below is the list of growth rates you will come across in the following chapters.

Time Complexity	Name	Example
1	Constant	Adding an element to the front of a linked list
$\log n$	Logarithmic	Finding an element in a sorted array
n	Linear	Finding an element in an unsorted array
$n \log n$	Linear Logarithmic	Sorting n items by 'divide-and-conquer' - Mergesort
n^2	Quadratic	Shortest path between two nodes in a graph
n^3	Cubic	Matrix Multiplication
2^n	Exponential	The Towers of Hanoi problem

1.12 Types of Analysis

To analyze the given algorithm, we need to know with which inputs the algorithm takes less time (performing well) and with which inputs the algorithm takes a long time. We have already seen that an algorithm can be represented in the form of an expression. That means we represent the algorithm with multiple expressions: one for the case where it takes less time and another for the case where it takes more time.

In general, the first case is called the *best case* and the second case is called the *worst case* for the algorithm. To analyze an algorithm we need some kind of syntax, and that forms the base for asymptotic analysis/notation. There are three types of analysis:

- **Worst case**
 - Defines the input for which the algorithm takes a long time (slowest time to complete).
 - Input is the one for which the algorithm runs the slowest.
- **Best case**
 - Defines the input for which the algorithm takes the least time (fastest time to complete).
 - Input is the one for which the algorithm runs the fastest.
- **Average case**
 - Provides a prediction about the running time of the algorithm.
 - Run the algorithm many times, using many different inputs that come from some distribution that generates these inputs, compute the total running time (by adding the individual times), and divide by the number of trials.
 - Assumes that the input is random.

$$\text{Lower Bound} \leq \text{Average Time} \leq \text{Upper Bound}$$

For a given algorithm, we can represent the best, worst and average cases in the form of expressions. As an example, let $f(n)$ be the function which represents the given algorithm.

$$\begin{aligned} f(n) &= n^2 + 500, \text{ for worst case} \\ f(n) &= n + 100n + 500, \text{ for best case} \end{aligned}$$

Similarly for the average case. The expression defines the inputs with which the algorithm takes the average running time (or memory).

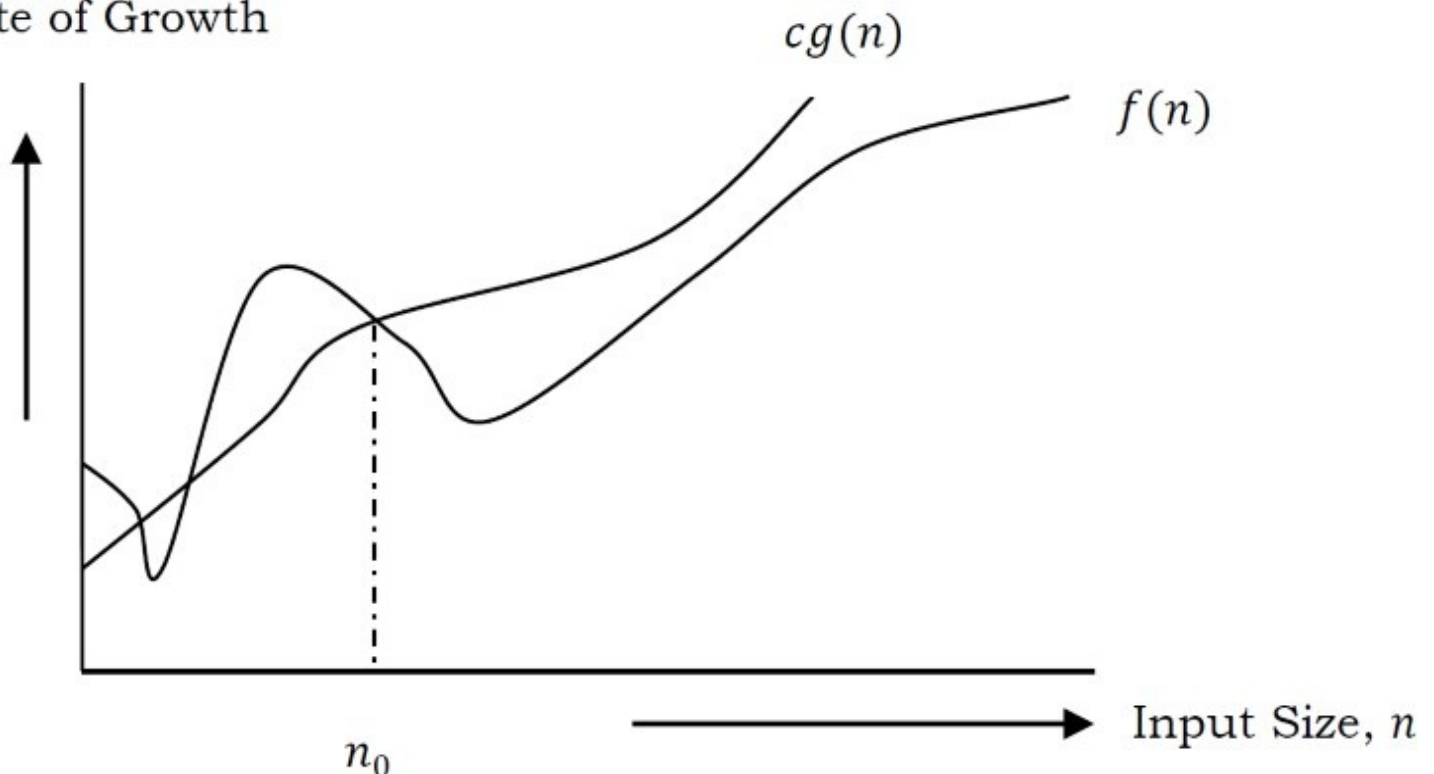
1.13 Asymptotic Notation

Having the expressions for the best, average and worst cases, for all three cases we need to identify the upper and lower bounds. To represent these upper and lower bounds, we need some kind of syntax, and that is the subject of the following discussion. Let us assume that the given algorithm is represented in the form of function $f(n)$.

1.14 Big-O Notation [Upper Bounding Function]

This notation gives the *tight* upper bound of the given function. Generally, it is represented as $f(n) = O(g(n))$. That means, at larger values of n , the upper bound of $f(n)$ is $g(n)$. For example, if $f(n) = n^4 + 100n^2 + 10n + 50$ is the given algorithm, then n^4 is $g(n)$. That means $g(n)$ gives the maximum rate of growth for $f(n)$ at larger values of n .

Rate of Growth



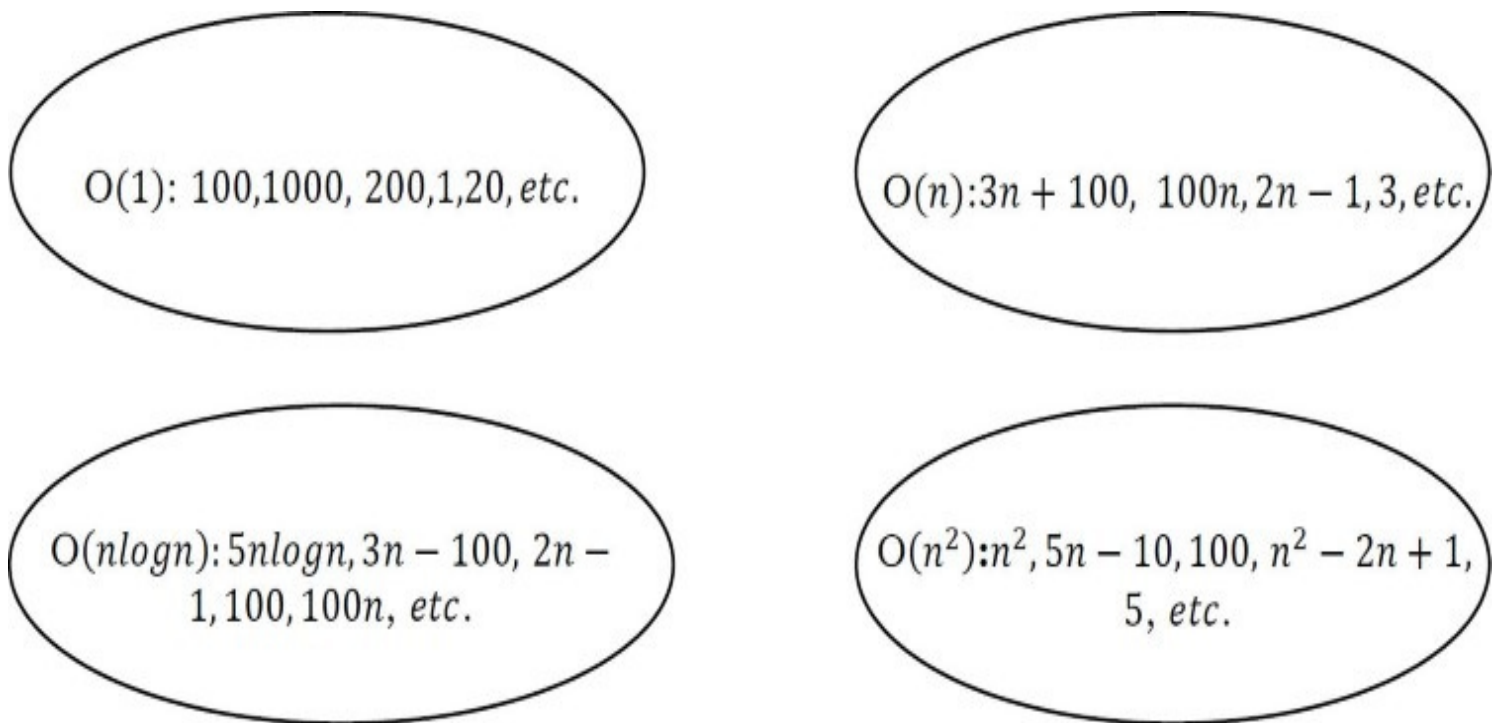
Let us see the O -notation with a little more detail. O -notation defined as $O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n > n_0\}$. $g(n)$ is an asymptotic tight upper bound for $f(n)$. Our objective is to give the smallest rate of growth $g(n)$ which is greater than or equal to the given algorithms' rate of growth $f(n)$.

Generally we discard lower values of n . That means the rate of growth at lower values of n is not important. In the figure, n_0 is the point from which we need to consider the rate of growth for a given algorithm. Below n_0 , the rate of growth could be different. n_0 is called threshold for the given function.

Big-O Visualization

$O(g(n))$ is the set of functions with smaller or the same order of growth as $g(n)$. For example; $O(n^2)$ includes $O(1)$, $O(n)$, $O(n \log n)$, etc.

Note: Analyze the algorithms at larger values of n only. What this means is, below n_0 we do not care about the rate of growth.



Big-O Examples

Example-1 Find upper bound for $f(n) = 3n + 8$

Solution: $3n + 8 \leq 4n$, for all $n \geq 8$
 $\therefore 3n + 8 = O(n)$ with $c = 4$ and $n_0 = 8$

Example-2 Find upper bound for $f(n) = n^2 + 1$

Solution: $n^2 + 1 \leq 2n^2$, for all $n \geq 1$
 $\therefore n^2 + 1 = O(n^2)$ with $c = 2$ and $n_0 = 1$

Example-3 Find upper bound for $f(n) = n^4 + 100n^2 + 50$

Solution: $n^4 + 100n^2 + 50 \leq 2n^4$, for all $n \geq 11$
 $\therefore n^4 + 100n^2 + 50 = O(n^4)$ with $c = 2$ and $n_0 = 11$

Example-4 Find upper bound for $f(n) = 2n^3 - 2n^2$

Solution: $2n^3 - 2n^2 \leq 2n^3$, for all $n > 1$
 $\therefore 2n^3 - 2n^2 = O(n^3)$ with $c = 2$ and $n_0 = 1$

Example-5 Find upper bound for $f(n) = n$

Solution: $n \leq n$, for all $n \geq 1$
 $\therefore n = O(n)$ with $c = 1$ and $n_0 = 1$

Example-6 Find upper bound for $f(n) = 410$

Solution: $410 \leq 410$, for all $n > 1$
 $\therefore 410 = O(1)$ with $c = 1$ and $n_0 = 1$

No Uniqueness?

There is no unique set of values for n_0 and c in proving the asymptotic bounds. Let us consider, $100n + 5 = O(n)$. For this function there are multiple n_0 and c values possible.

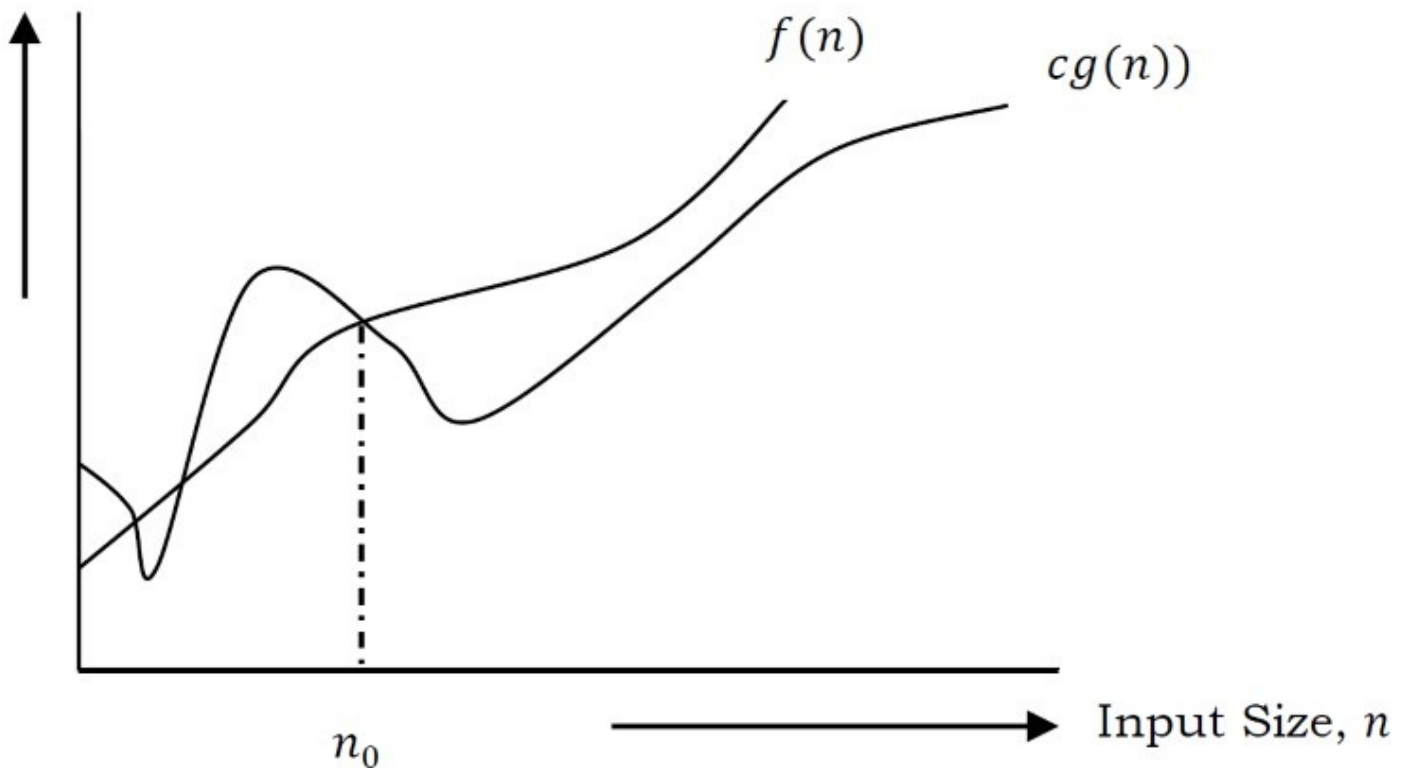
Solution1: $100n + 5 \leq 100n + n = 101n \leq 101n$, for all $n \geq 5$, $n_0 = 5$ and $c = 101$ is a solution.

Solution2: $100n + 5 \leq 100n + 5n = 105n \leq 105n$, for all $n > 1$, $n_0 = 1$ and $c = 105$ is also a solution.

1.15 Omega-Q Notation [Lower Bounding Function]

Similar to the O discussion, this notation gives the tighter lower bound of the given algorithm and we represent it as $f(n) = \Omega(g(n))$. That means, at larger values of n , the tighter lower bound of $f(n)$ is $g(n)$. For example, if $f(n) = 100n^2 + 10n + 50$, $g(n)$ is $\Omega(n^2)$.

Rate of Growth



The Ω notation can be defined as $\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$. $g(n)$ is an asymptotic tight lower bound for $f(n)$. Our objective is to give the largest rate of growth $g(n)$ which is less than or equal to the given algorithm's rate of growth $f(n)$.

Ω Examples

Example-1 Find lower bound for $f(n) = 5n^2$.

Solution: $\exists c, n_0$ Such that: $0 \leq cn^2 \leq 5n^2 \Rightarrow cn^2 \leq 5n^2 \Rightarrow c = 5$ and $n_0 = 1$
 $\therefore 5n^2 = \Omega(n^2)$ with $c = 5$ and $n_0 = 1$

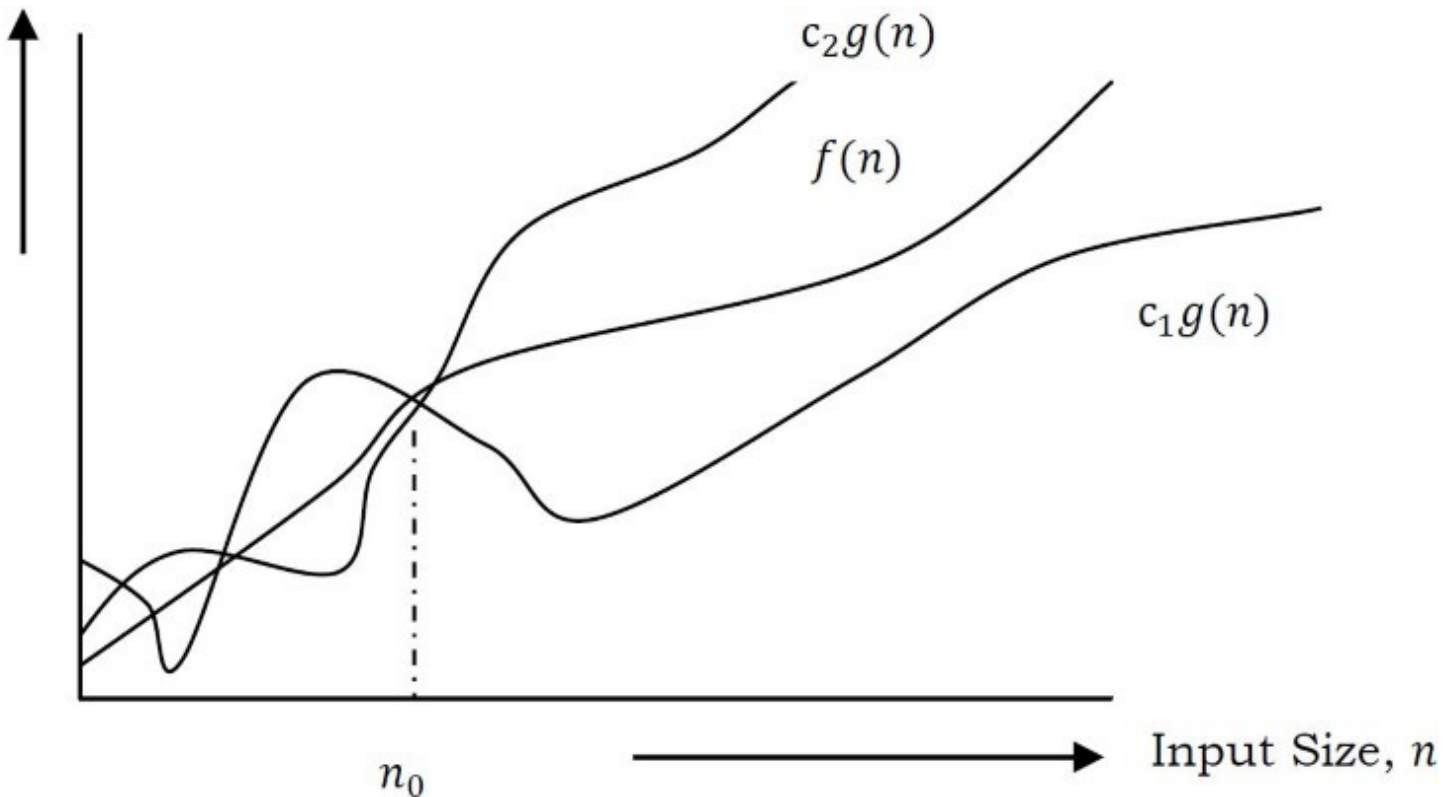
Example-2 Prove $f(n) = 100n + 5 \neq \Omega(n^2)$.

Solution: $\exists c, n_0$ Such that: $0 \leq cn^2 \leq 100n + 5$
 $100n + 5 \leq 100n + 5n (\forall n \geq 1) = 105n$
 $cn^2 \leq 105n \Rightarrow n(cn - 105) \leq 0$
 Since n is positive $\Rightarrow cn - 105 \leq 0 \Rightarrow n \leq 105/c$
 $\Rightarrow$ Contradiction: n cannot be smaller than a constant

Example-3 $2n = Q(n)$, $n^3 = Q(n^3)$, $= O(\log n)$.

1.16 Theta- Θ Notation [Order Function]

Rate of Growth



This notation decides whether the upper and lower bounds of a given function (algorithm) are the same. The average running time of an algorithm is always between the lower bound and the upper bound. If the upper bound (O) and lower bound (Ω) give the same result, then the Θ notation will also have the same rate of growth.

As an example, let us assume that $f(n) = 10n + n$ is the expression. Then, its tight upper bound $g(n)$ is $O(n)$. The rate of growth in the best case is $g(n) = O(n)$.

In this case, the rates of growth in the best case and worst case are the same. As a result, the average case will also be the same. For a given function (algorithm), if the rates of growth (bounds) for O and Ω are not the same, then the rate of growth for the Θ case may not be the same. In this case, we need to consider all possible time complexities and take the average of those (for example, for a quick sort average case, refer to the [Sorting](#) chapter).

Now consider the definition of Θ notation. It is defined as $\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$. $g(n)$ is an asymptotic tight bound for $f(n)$. $\Theta(g(n))$ is the set of functions with the same order of growth as $g(n)$.

Θ Examples

Example 1 Find Θ bound for $f(n) = \frac{n^2}{2} - \frac{n}{2}$

Solution: $\frac{n^2}{5} \leq \frac{n^2}{2} - \frac{n}{2} \leq n^2$ for all, $n \geq 2$
 $\therefore \frac{n^2}{2} - \frac{n}{2} = \Theta(n^2)$ with $c_1 = 1/5, c_2 = 1$ and $n_0 = 2$

Example 2 Prove $n \neq \Theta(n^2)$

Solution: $c_1 n^2 \leq n \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq 1/c_1$
 $\therefore n \neq \Theta(n^2)$

Example 3 Prove $6n^3 \neq \Theta(n^2)$

Solution: $c_1 n^2 \leq 6n^3 \leq c_2 n^2 \Rightarrow$ only holds for: $n \leq c_2/6$
 $\therefore 6n^3 \neq \Theta(n^2)$

Example 4 Prove $n \neq \Theta(\log n)$

Solution: $c_1 \log n \leq n \leq c_2 \log n \Rightarrow c_2 \geq \frac{n}{\log n}, \forall n \geq n_0 - \text{Impossible}$

1.17 Important Notes

For analysis (best case, worst case and average), we try to give the upper bound (O) and lower bound (Ω) and average running time (Θ). From the above examples, it should also be clear that, for a given function (algorithm), getting the upper bound (O) and lower bound (Ω) and average running time (Θ) may not always be possible. For example, if we are discussing the best case of an algorithm, we try to give the upper bound (O) and lower bound (Ω) and average running time (Θ).

In the remaining chapters, we generally focus on the upper bound (O) because knowing the lower bound (Ω) of an algorithm is of no practical importance, and we use the Θ notation if the upper bound (O) and lower bound (Ω) are the same.

1.18 Why is it called Asymptotic Analysis?

From the discussion above (for all three notations: worst case, best case, and average case), we can easily understand that, in every case for a given function $f(n)$ we are trying to find another function $g(n)$ which approximates $f(n)$ at higher values of n . That means $g(n)$ is also a curve which approximates $f(n)$ at higher values of n .

In mathematics we call such a curve an *asymptotic curve*. In other terms, $g(n)$ is the asymptotic

curve for $f(n)$. For this reason, we call algorithm analysis *asymptotic analysis*.

1.19 Guidelines for Asymptotic Analysis

There are some general rules to help us determine the running time of an algorithm.

- 1) **Loops:** The running time of a loop is, at most, the running time of the statements inside the loop (including tests) multiplied by the number of iterations.

```
// executes n times
for (i=1; i<=n; i++)
    m = m + 2; // constant time, c
```

Total time = a constant $c \times n = c n = O(n)$.

- 2) **Nested loops:** Analyze from the inside out. Total running time is the product of the sizes of all the loops.

```
//outer loop executed n times
for (i=1; i<=n; i++) {
    // inner loop executes n times
    for (j=1; j<=n; j++)
        k = k+1; //constant time
}
```

Total time = $c \times n \times n = cn^2 = O(n^2)$.

- 3) **Consecutive statements:** Add the time complexities of each statement.

```

x = x + 1; //constant time
// executes n times
for (i=1; i<=n; i++)
    m = m + 2; //constant time
//outer loop executes n times
for (i=1; i<=n; i++) {
    //inner loop executed n times
    for (j=1; j<=n; j++)
        k = k+1; //constant time
}

```

Total time = $c_0 + c_1n + c_2n^2 = O(n^2)$.

- 4) **If-then-else statements:** Worst-case running time: the test, plus *either* the *then* part or the *else* part (whichever is the larger).

```

//test: constant
if(length( ) == 0 ) {
    return false; //then part: constant
}
else { // else part: (constant + constant) * n
    for (int n = 0; n < length( ); n++) {
        // another if : constant + constant (no else part)
        if(!list[n].equals(otherList.list[n]))
            //constant
            return false;
    }
}

```

Total time = $c_0 + c_1 + (c_2 + c_3) * n = O(n)$.

- 5) **Logarithmic complexity:** An algorithm is $O(\log n)$ if it takes a constant time to cut the problem size by a fraction (usually by $\frac{1}{2}$). As an example let us consider the following program:

```

for (i=1; i<=n;)
    i = i*2;

```

If we observe carefully, the value of i is doubling every time. Initially $i = 1$, in next step $i = 2$, and in subsequent steps $i = 4, 8$ and so on. Let us assume that the loop is executing some k times. At k^{th} step $2^k = n$, and at $(k + 1)^{th}$ step we come out of the *loop*. Taking logarithm on both sides, gives

$$\log(2^k) = \log n$$

$$k \log 2 = \log n$$

$$k = \log n \quad // \text{if we assume base-2}$$

Total time = $O(\log n)$.

Note: Similarly, for the case below, the worst case rate of growth is $O(\log n)$. The same discussion holds good for the decreasing sequence as well.

```
for (i=n; i>=1; i = i/2;
```

Another example: binary search (finding a word in a dictionary of n pages)

- Look at the center point in the dictionary
- Is the word towards the left or right of center?
- Repeat the process with the left or right part of the dictionary until the word is found.

1.20 Simplifying properties of asymptotic notations

- Transitivity: $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$. Valid for O and Ω as well.
- Reflexivity: $f(n) = \Theta(f(n))$. Valid for O and Ω .
- Symmetry: $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$.
- Transpose symmetry: $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$.
- If $f(n)$ is in $O(kg(n))$ for any constant $k > 0$, then $f(n)$ is in $O(g(n))$.
- If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $(f_1 + f_2)(n)$ is in $O(\max(g_1(n), g_2(n)))$.
- If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $f_1(n) f_2(n)$ is in $O(g_1(n) g_2(n))$.

1.21 Commonly used Logarithms and Summations

Logarithms

$$\log x^y = y \log x$$

$$\log xy = \log x + \log y$$

$$\log \log n = \log(\log n)$$

$$a^{\log_b x} = x^{\log_b a}$$

$$\log n = \log_{10}^n$$

$$\log^k n = (\log n)^k$$

$$\log \frac{x}{y} = \log x - \log y$$

$$\log_b^x = \frac{\log_a^x}{\log_a^b}$$

Arithmetic series

$$\sum_{k=1}^n k = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Geometric series

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1} (x \neq 1)$$

Harmonic series

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \log n$$

Other important formulae

$$\sum_{k=1}^n \log k \approx n \log n$$

$$\sum_{k=1}^n k^p = 1^p + 2^p + \dots + n^p \approx \frac{1}{p+1} n^{p+1}$$

1.22 Master Theorem for Divide and Conquer Recurrences

All divide and conquer algorithms (also discussed in detail in the *Divide and Conquer* chapter) divide the problem into sub-problems, each of which is part of the original problem, and then perform some additional work to compute the final answer. As an example, a merge sort algorithm [for details, refer to [Sorting](#) chapter] operates on two sub-problems, each of which is half the size of the original, and then performs $O(n)$ additional work for merging. This gives the

running time equation:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

The following theorem can be used to determine the running time of divide and conquer algorithms. For a given program (algorithm), first we try to find the recurrence relation for the problem. If the recurrence is of the below form then we can directly give the answer without fully solving it. If the recurrence is of the form $T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k \log^p n)$, where $a \geq 1, b > 1, k \geq 0$ and p is a real number, then:

- 1) If $a > b^k$, then $T(n) = \Theta(n^{\log_b a})$
- 2) If $a = b^k$
 - a. If $p > -1$, then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
 - b. If $p = -1$, then $T(n) = \Theta(n^{\log_b a} \log \log n)$
 - c. If $p < -1$, then $T(n) = \Theta(n^{\log_b a})$
- 3) If $a < b^k$
 - a. If $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$
 - b. If $p < 0$, then $T(n) = O(n^k)$

1.23 Divide and Conquer Master Theorem: Problems & Solutions

For each of the following recurrences, give an expression for the runtime $T(n)$ if the recurrence can be solved with the Master Theorem. Otherwise, indicate that the Master Theorem does not apply.

Problem-1 $T(n) = 3T(n/2) + n^2$

Solution: $T(n) = 3T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-2 $T(n) = 4T(n/2) + n^2$

Solution: $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2 \log n)$ (Master Theorem Case 2.a)

Problem-3 $T(n) = T(n/2) + n^2$

Solution: $T(n) = T(n/2) + n^2 \Rightarrow \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-4 $T(n) = 2^n T(n/2) + n^n$

Solution: $T(n) = 2^n T(n/2) + n^n \Rightarrow$ Does not apply (a is not constant)

Problem-5 $T(n) = 16T(n/4) + n$

Solution: $T(n) = 16T(n/4) + n \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-6 $T(n) = 2T(n/2) + n \log n$

Solution: $T(n) = 2T(n/2) + n \log n \Rightarrow T(n) = \Theta(n \log^2 n)$ (Master Theorem Case 2.a)

Problem-7 $T(n) = 2T(n/2) + n/\log n$

Solution: $T(n) = 2T(n/2) + n/\log n \Rightarrow T(n) = \Theta(n \log \log n)$ (Master Theorem Case 2. b)

Problem-8 $T(n) = 2T(n/4) + n^{0.51}$

Solution: $T(n) = 2T(n/4) + n^{0.51} \Rightarrow T(n) = \Theta(n^{0.51})$ (Master Theorem Case 3.b)

Problem-9 $T(n) = 0.5T(n/2) + 1/n$

Solution: $T(n) = 0.5T(n/2) + 1/n \Rightarrow$ Does not apply ($a < 1$)

Problem-10 $T(n) = 6T(n/3) + n^2 \log n$

Solution: $T(n) = 6T(n/3) + n^2 \log n \Rightarrow T(n) = \Theta(n^2 \log n)$ (Master Theorem Case 3.a)

Problem-11 $T(n) = 64T(n/8) - n^2 \log n$

Solution: $T(n) = 64T(n/8) - n^2 \log n \Rightarrow$ Does not apply (function is not positive)

Problem-12 $T(n) = 7T(n/3) + n^2$

Solution: $T(n) = 7T(n/3) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 3.as)

Problem-13 $T(n) = 4T(n/2) + \log n$

Solution: $T(n) = 4T(n/2) + \log n \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-14 $T(n) = 16T(n/4) + n!$

Solution: $T(n) = 16T(n/4) + n! \Rightarrow T(n) = \Theta(n!)$ (Master Theorem Case 3.a)

Problem-15 $T(n) = \sqrt{2}T(n/2) + \log n$

Solution: $T(n) = \sqrt{2}T(n/2) + \log n \Rightarrow T(n) = \Theta(\sqrt{n})$ (Master Theorem Case 1)

Problem-16 $T(n) = 3T(n/2) + n$

Solution: $T(n) = 3T(n/2) + n \Rightarrow T(n) = \Theta(n^{\log 3})$ (Master Theorem Case 1)

Problem-17 $T(n) = 3T(n/3) + \sqrt{n}$

Solution: $T(n) = 3T(n/3) + \sqrt{n} \Rightarrow T(n) = \Theta(n)$ (Master Theorem Case 1)

Problem-18 $T(n) = 4T(n/2) + cn$

Solution: $T(n) = 4T(n/2) + cn \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 1)

Problem-19 $T(n) = 3T(n/4) + n \log n$

Solution: $T(n) = 3T(n/4) + n \log n \Rightarrow T(n) = \Theta(n \log n)$ (Master Theorem Case 3.a)

Problem-20 $T(n) = 3T(n/3) + n/2$

Solution: $T(n) = 3T(n/3) + n/2 \Rightarrow T(n) = \Theta(n \log n)$ (Master Theorem Case 2.a)

1.24 Master Theorem for Subtract and Conquer Recurrences

Let $T(n)$ be a function defined on positive n , and having the property

$$T(n) = \begin{cases} c, & \text{if } n \leq 1 \\ aT(n-b) + f(n), & \text{if } n > 1 \end{cases}$$

for some constants $c, a > 0, b \geq 0, k \geq 0$, and function $f(n)$. If $f(n)$ is in $O(n^k)$, then

$$T(n) = \begin{cases} O(n^k), & \text{if } a < 1 \\ O(n^{k+1}), & \text{if } a = 1 \\ O\left(n^k a^{\frac{n}{b}}\right), & \text{if } a > 1 \end{cases}$$

1.25 Variant of Subtraction and Conquer Master Theorem

The solution to the equation $T(n) = T(\alpha n) + T((1 - \alpha)n) + \beta n$, where $0 < \alpha < 1$ and $\beta > 0$ are constants, is $O(n \log n)$.

1.26 Method of Guessing and Confirming

Now, let us discuss a method which can be used to solve any recurrence. The basic idea behind this method is:

guess the answer; and then prove it correct by induction.

In other words, it addresses the question: What if the given recurrence doesn't seem to match with any of these (master theorem) methods? If we guess a solution and then try to verify our guess inductively, usually either the proof will succeed (in which case we are done), or the proof will fail (in which case the failure will help us refine our guess).

As an example, consider the recurrence $T(n) = \sqrt{n} T(\sqrt{n}) + n$. This doesn't fit into the form required by the Master Theorems. Carefully observing the recurrence gives us the impression that it is similar to the divide and conquer method (dividing the problem into $\sqrt{n}$ subproblems each with size $\sqrt{n}$). As we can see, the size of the subproblems at the first level of recursion is n . So, let us guess that $T(n) = O(n \log n)$, and then try to prove that our guess is correct.

Let's start by trying to prove an *upper* bound $T(n) < cn \log n$:

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\leq \sqrt{n} \cdot c\sqrt{n} \log \sqrt{n} + n \\
&= n \cdot c \log \sqrt{n} + n \\
&= n \cdot c \cdot \frac{1}{2} \cdot \log n + n \\
&\leq cn \log n
\end{aligned}$$

The last inequality assumes only that $1 \leq c \cdot \frac{1}{2} \cdot \log n$. This is correct if n is sufficiently large and for any constant c , no matter how small. From the above proof, we can see that our guess is correct for the upper bound. Now, let us prove the *lower* bound for this recurrence.

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\geq \sqrt{n} \cdot k \sqrt{n} \log \sqrt{n} + n \\
&= n \cdot k \log \sqrt{n} + n \\
&= n \cdot k \cdot \frac{1}{2} \cdot \log n + n \\
&\geq kn \log n
\end{aligned}$$

The last inequality assumes only that $1 \geq k \cdot \frac{1}{2} \cdot \log n$. This is incorrect if n is sufficiently large and for any constant k . From the above proof, we can see that our guess is incorrect for the lower bound.

From the above discussion, we understood that $\Theta(n \log n)$ is too big. How about $\Theta(n)$? The lower bound is easy to prove directly:

$$T(n) = \sqrt{n} T(\sqrt{n}) + n \geq n$$

Now, let us prove the upper bound for this $\Theta(n)$.

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\leq \sqrt{n} \cdot c \cdot \sqrt{n} + n \\
&= n \cdot c + n \\
&= n (c + 1) \\
&\not\leq cn
\end{aligned}$$

From the above induction, we understood that $\Theta(n)$ is too small and $\Theta(n \log n)$ is too big. So, we need something bigger than n and smaller than $n \log n$. How about $n\sqrt{\log n}$?

Proving the upper bound for $n\sqrt{\log n}$:

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\leq \sqrt{n} \cdot c \cdot \sqrt{n} \sqrt{\log \sqrt{n}} + n \\
&= n \cdot c \cdot \frac{1}{\sqrt{2}} \log \sqrt{n} + n \\
&\leq cn \log \sqrt{n}
\end{aligned}$$

Proving the lower bound for $n\sqrt{\log n}$:

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\geq \sqrt{n} \cdot k \cdot \sqrt{n} \sqrt{\log \sqrt{n}} + n \\
&= n \cdot k \cdot \frac{1}{\sqrt{2}} \log \sqrt{n} + n \\
&\nless kn \log \sqrt{n}
\end{aligned}$$

The last step doesn't work. So, $\Theta(n\sqrt{\log n})$ doesn't work. What else is between n and $n \log n$? How about $n \log \log n$? Proving upper bound for $n \log \log n$:

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\leq \sqrt{n} \cdot c \cdot \sqrt{n} \log \log \sqrt{n} + n \\
&= n \cdot c \cdot \log \log n - c \cdot n + n \\
&\leq cn \log \log n, \text{ if } c \geq 1
\end{aligned}$$

Proving lower bound for $n \log \log n$:

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&\geq \sqrt{n} \cdot k \cdot \sqrt{n} \log \log \sqrt{n} + n \\
&= n \cdot k \cdot \log \log n - k \cdot n + n \\
&\geq kn \log \log n, \text{ if } k \leq 1
\end{aligned}$$

From the above proofs, we can see that $T(n) \leq cn \log \log n$, if $c \geq 1$ and $T(n) \geq kn \log \log n$, if $k \leq 1$. Technically, we're still missing the base cases in both proofs, but we can be fairly confident at this point that $T(n) = \Theta(n \log \log n)$.

1.27 Amortized Analysis

Amortized analysis refers to determining the time-averaged running time for a sequence of operations. It is different from average case analysis, because amortized analysis does not make any assumption about the distribution of the data values, whereas average case analysis assumes the data are not “bad” (e.g., some sorting algorithms do well *on average* over all input orderings but very badly on certain input orderings). That is, amortized analysis is a worst-case analysis, but for a sequence of operations rather than for individual operations.

The motivation for amortized analysis is to better understand the running time of certain techniques, where standard worst case analysis provides an overly pessimistic bound. Amortized analysis generally applies to a method that consists of a sequence of operations, where the vast majority of the operations are cheap, but some of the operations are expensive. If we can show that the expensive operations are particularly rare we can *change them* to the cheap operations, and only bound the cheap operations.

The general approach is to assign an artificial cost to each operation in the sequence, such that the total of the artificial costs for the sequence of operations bounds the total of the real costs for the sequence. This artificial cost is called the amortized cost of an operation. To analyze the running time, the amortized cost thus is a correct way of understanding the overall running time – but note that particular operations can still take longer so it is not a way of bounding the running time of any individual operation in the sequence.

When one event in a sequence affects the cost of later events:

- One particular task may be expensive.
- But it may leave data structure in a state that the next few operations become easier.

Example: Let us consider an array of elements from which we want to find the k^{th} smallest element. We can solve this problem using sorting. After sorting the given array, we just need to return the k^{th} element from it. The cost of performing the sort (assuming comparison based sorting algorithm) is $O(n \log n)$. If we perform n such selections then the average cost of each selection is $O(n \log n / n) = O(\log n)$. This clearly indicates that sorting once is reducing the complexity of subsequent operations.

1.28 Algorithms Analysis: Problems & Solutions

Note: From the following problems, try to understand the cases which have different complexities ($O(n)$, $O(\log n)$, $O(\log \log n)$ etc.).

Problem-21 Find the complexity of the below recurrence:

$$T(n) = \begin{cases} 3T(n-1), & \text{if } n > 0, \\ 1, & \text{otherwise} \end{cases}$$

Solution: Let us try solving this function with substitution.

$$T(n) = 3T(n - 1)$$

$$T(n) = 3(3T(n - 2)) = 3^2T(n - 2)$$

$$T(n) = 3^2(3T(n - 3))$$

.

.

$$T(n) = 3^nT(n - n) = 3^nT(0) = 3^n$$

This clearly shows that the complexity of this function is $O(3^n)$.

Note: We can use the *Subtraction and Conquer* master theorem for this problem.

Problem-22 Find the complexity of the below recurrence:

$$T(n) = \begin{cases} 2T(n - 1) - 1, & \text{if } n > 0, \\ 1, & \text{otherwise} \end{cases}$$

Solution: Let us try solving this function with substitution.

$$T(n) = 2T(n - 1) - 1$$

$$T(n) = 2(2T(n - 2) - 1) - 1 = 2^2T(n - 2) - 2 - 1$$

$$T(n) = 2^2(2T(n - 3) - 2 - 1) - 1 = 2^3T(n - 4) - 2^2 - 2^1 - 2^0$$

$$T(n) = 2^nT(n - n) - 2^{n-1} - 2^{n-2} - 2^{n-3} \dots 2^2 - 2^1 - 2^0$$

$$T(n) = 2^n - 2^{n-1} - 2^{n-2} - 2^{n-3} \dots 2^2 - 2^1 - 2^0$$

$$T(n) = 2^n - (2^n - 1) \text{ [note: } 2^{n-1} + 2^{n-2} + \dots + 2^0 = 2^n]$$

$$T(n) = 1$$

$\therefore$ Time Complexity is $O(1)$. Note that while the recurrence relation looks exponential, the solution to the recurrence relation here gives a different result.

Problem-23 What is the running time of the following function?

```

void Function(int n) {
    int i=1, s=1;
    while( s <= n) {
        i++;
        s= s+i;
        printf("*");
    }
}

```

Solution: Consider the comments in the below function:

```

void Function (int n) {
    int i=1, s=1;
    // s is increasing not at rate 1 but i
    while( s <= n) {
        i++;
        s= s+i;
        printf("*");
    }
}

```

We can define the 's' terms according to the relation $s_i = s_{i-1} + i$. The value of 's' increases by 1 for each iteration. The value contained in 's' at the i^{th} iteration is the sum of the first 'i' positive integers. If k is the total number of iterations taken by the program, then the *while* loop terminates if:

$$1 + 2 + \dots + k = \frac{k(k+1)}{2} > n \Rightarrow k = O(\sqrt{n}).$$

Problem-24 Find the complexity of the function given below.

```

void function(int n) {
    int i, count =0;
    for(i=1; i*i<=n; i++)
        count++;
}

```

Solution:

```

void function(int n) {
    int i, count = 0;
    for(i=1; i*i<=n; i++)
        count++;
}

```

In the above-mentioned function the loop will end, if $i^2 > n \Rightarrow T(n) = O(\sqrt{n})$. This is similar to [Problem-23](#).

Problem-25 What is the complexity of the program given below:

```

void function(int n) {
    int i, j, k , count = 0;
    for(i=n/2; i<=n; i++)
        for(j=1; j + n/2<=n; j= j+1)
            for(k=1; k<=n; k= k * 2)
                count++;
}

```

Solution: Consider the comments in the following function.

```

void function(int n) {
    int i, j, k , count = 0;
    //outer loop execute n/2 times
    for(i=n/2; i<=n; i++)
        //middle loop executes n/2 times
        for(j=1; j + n/2<=n; j= j+1)
            //inner loop execute logn times
            for(k=1; k<=n; k= k * 2)
                count++;
}

```

The complexity of the above function is $O(n^2 \log n)$.

Problem-26 What is the complexity of the program given below:

```

void function(int n) {
    int i, j, k , count =0;
    for(i=n/2; i<=n; i++)
        for(j=1; j<=n; j= 2 * j)
            for(k=1; k<=n; k= k * 2)
                count++;
}

```

Solution: Consider the comments in the following function.

```

void function(int n) {
    int i, j, k , count =0;
    //outer loop execute n/2 times
    for(i=n/2; i<=n; i++)
        //middle loop executes logn times
        for(j=1; j<=n; j= 2 * j)
            //inner loop execute logn times
            for(k=1; k<=n; k= k*2)
                count++;
}

```

The complexity of the above function is $O(n\log^2n)$.

Problem-27 Find the complexity of the program below.

```

function( int n ) {
    if(n == 1) return;
    for(int i = 1 ; i <= n ; i + + ) {
        for(int j= 1 ; j <= n ; j + + ) {
            printf("*" );
            break;
        }
    }
}

```

Solution: Consider the comments in the function below.

```

function( int n ) {
    //constant time
    if( n == 1 ) return;
    //outer loop execute n times
    for(int i = 1 ; i <= n ; i + + ) {
        // inner loop executes only time due to break statement.
        for(int j = 1 ; j <= n ; j + + ) {
            printf("*");
            break;
        }
    }
}

```

The complexity of the above function is $O(n)$. Even though the inner loop is bounded by n , due to the break statement it is executing only once.

Problem-28 Write a recursive function for the running time $T(n)$ of the function given below. Prove using the iterative method that $T(n) = \Theta(n^3)$.

```

function( int n ) {
    if( n == 1 ) return;
    for(int i = 1 ; i <= n ; i + + )
        for(int j = 1 ; j <= n ; j + + )
            printf("*");
    function( n-3 );
}

```

Solution: Consider the comments in the function below:

```

function (int n) {
    //constant time
    if( n == 1 ) return;
    //outer loop execute n times
    for(int i = 1 ; i <= n ; i ++ )
        //inner loop executes n times
        for(int j = 1 ; j <= n ; j ++ )
            //constant time
            printf("#" );
        function( n-3 );
}

```

The recurrence for this code is clearly $T(n) = T(n - 3) + cn^2$ for some constant $c > 0$ since each call prints out n^2 asterisks and calls itself recursively on $n - 3$. Using the iterative method we get: $T(n) = T(n - 3) + cn^2$. Using the *Subtraction and Conquer* master theorem, we get $T(n) = \Theta(n^3)$.

Problem-29 Determine Θ bounds for the recurrence relation: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

Solution: Using Divide and Conquer master theorem, we get $O(n \log^2 n)$.

Problem-30 Determine Θ bounds for the recurrence:

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n$$

Solution: Substituting in the recurrence equation, we get:
 $T(n) \leq c1 * \frac{n}{2} + c2 * \frac{n}{4} + c3 * \frac{n}{8} + cn \leq k * n$, where k is a constant. This clearly says $\Theta(n)$.

Problem-31 Determine Θ bounds for the recurrence relation: $T(n) = T(\lfloor n/2 \rfloor) + 7$.

Solution: Using Master Theorem we get: $\Theta(\log n)$.

Problem-32 Prove that the running time of the code below is $\Omega(\log n)$.

```

void Read(int n) {
    int k = 1;
    while( k < n )
        k = 3*k;
}

```

Solution: The *while* loop will terminate once the value of ' k ' is greater than or equal to the value of ' n '. In each iteration the value of ' k ' is multiplied by 3. If i is the number of iterations, then ' k ' has the value of 3^i after i iterations. The loop is terminated upon reaching i iterations when $3^i \geq n$

$\leftrightarrow i \geq \log_3 n$, which shows that $i = \Omega(\log n)$.

Problem-33 Solve the following recurrence.

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ T(n-1) + n(n-1), & \text{if } n \geq 2 \end{cases}$$

Solution: By iteration:

$$T(n) = T(n-2) + (n-1)(n-2) + n(n-1)$$

...

$$T(n) = T(1) + \sum_{i=1}^n i(i-1)$$

$$T(n) = T(1) + \sum_{i=1}^n i^2 - \sum_{i=1}^n i$$

$$T(n) = 1 + \frac{n((n+1)(2n+1))}{6} - \frac{n(n+1)}{2}$$

$$T(n) = \Theta(n^3)$$

Note: We can use the *Subtraction and Conquer* master theorem for this problem.

Problem-34 Consider the following program:

```
Fib[n]
if(n==0) then return 0
else if(n==1) then return 1
else return Fib[n-1]+Fib[n-2]
```

Solution: The recurrence relation for the running time of this program is: $T(n) = T(n-1) + T(n-2) + c$. Note $T(n)$ has two recurrence calls indicating a binary tree. Each step recursively calls the program for n reduced by 1 and 2, so the depth of the recurrence tree is $O(n)$. The number of leaves at depth n is 2^n since this is a full binary tree, and each leaf takes at least $O(1)$ computations for the constant factor. Running time is clearly exponential in n and it is $O(2^n)$.

Problem-35 Running time of following program?

```

function(n) {
    for(int i = 1; i <= n ; i + + )
        for(int j = 1 ; j <= n ; j+ = i )
            printf(" * ");
}

```

Solution: Consider the comments in the function below:

```

function (n) {
    //this loop executes n times
    for(int i = 1; i <= n ; i + + )
        //this loop executes j times with j increase by the rate of i
        for(int j = 1 ; j <= n ; j+ = i )
            printf( " * " );
}

```

In the above code, inner loop executes n/i times for each value of i . Its running time is $n \times (\sum_{i=1}^n n/i) = O(n \log n)$.

Problem-36 What is the complexity of $\sum_{i=1}^n \log i$?

Solution: Using the logarithmic property, $\log xy = \log x + \log y$, we can see that this problem is equivalent to

$$\sum_{i=1}^n \log i = \log 1 + \log 2 + \dots + \log n = \log(1 \times 2 \times \dots \times n) = \log(n!) \leq \log(n^n) \leq n \log n$$

This shows that the time complexity = $O(n \log n)$.

Problem-37 What is the running time of the following recursive function (specified as a function of the input value n)? First write the recurrence formula and then find its complexity.

```

function(int n) {
    if(n <= 1) return;
    for (int i=1 ; i <= 3; i++ )
        f( $\lceil \frac{n}{3} \rceil$ );
}

```

Solution: Consider the comments in the below function:

```

function (int n) {
    //constant time
    if(n <= 1) return;
    //this loop executes with recursive loop of  $\frac{n}{3}$  value
    for (int i=1 ; i <= 3; i++ )
        f( $\frac{n}{3}$ );
}

```

We can assume that for asymptotical analysis $k = \lceil k \rceil$ for every integer $k \geq 1$. The recurrence for this code is $T(n) = 3T(\frac{n}{3}) + \Theta(1)$. Using master theorem, we get $T(n) = \Theta(n)$.

Problem-38 What is the running time of the following recursive function (specified as a function of the input value n)? First write a recurrence formula, and show its solution using induction.

```

function(int n) {
    if(n <= 1) return;

    for (int i=1 ; i <= 3 ; i++ )
        function (n - 1).
}

```

Solution: Consider the comments in the function below:

```

function (int n) {
    //constant time
    if(n <= 1) return;
    //this loop executes 3 times with recursive call of n-1 value
    for (int i=1 ; i <= 3 ; i++ )
        function (n - 1).
}

```

The *if* statement requires constant time [$O(1)$]. With the *for* loop, we neglect the loop overhead and only count three times that the function is called recursively. This implies a time complexity recurrence:

$$\begin{aligned}
 T(n) &= c, \text{ if } n \leq 1; \\
 &= c + 3T(n - 1), \text{ if } n > 1.
 \end{aligned}$$

Using the *Subtraction and Conquer* master theorem, we get $T(n) = \Theta(3^n)$.

Problem-39 Write a recursion formula for the running time $T(n)$ of the function whose code is below.

```
function (int n) {
    if(n <= 1) return;
    for(int i = 1; i < n; i++)
        printf(" * ");
    function ( 0.8n ) ;
}
```

Solution: Consider the comments in the function below:

```
function (int n) {
    if(n <= 1) return; //constant time
    // this loop executes n times with constant time loop
    for(int i = 1; i < n; i++)
        printf(" * ");
    //recursive call with 0.8n
    function ( 0.8n ) ;
}
```

The recurrence for this piece of code is $T(n) = T(.8n) + O(n) = T(4/5n) + O(n) = 4/5 T(n) + O(n)$. Applying master theorem, we get $T(n) = O(n)$.

Problem-40 Find the complexity of the recurrence: $T(n) = 2T(\sqrt{n}) + \log n$

Solution: The given recurrence is not in the master theorem format. Let us try to convert this to the master theorem format by assuming $n = 2^m$. Applying the logarithm on both sides gives, $\log n = m \log 2 \Rightarrow m = \log n$. Now, the given function becomes:

$$T(n) = T(2^m) = 2T(\sqrt{2^m}) + m = 2T\left(2^{\frac{m}{2}}\right) + m.$$

To make it simple we assume $S(m) = T(2^m) \Rightarrow S\left(\frac{m}{2}\right) = T\left(2^{\frac{m}{2}}\right) \Rightarrow S(m) = 2S\left(\frac{m}{2}\right) + m.$

Applying the master theorem format would result in $S(m) = O(m \log m)$. If we substitute $m = \log n$ back, $T(n) = S(\log n) = O((\log n) \log \log n)$.

Problem-41 Find the complexity of the recurrence: $T(n) = T(\sqrt{n}) + 1$

Solution: Applying the logic of [Problem-40](#) gives $S(m) = S\left(\frac{m}{2}\right) + 1$. Applying the master

theorem would result in $S(m) = O(\log m)$. Substituting $m = \log n$, gives $T(n) = S(\log n) = O(\log \log n)$.

Problem-42 Find the complexity of the recurrence: $T(n) = 2T(\sqrt{n}) + 1$

Solution: Applying the logic of [Problem-40](#) gives: $S(m) = 2S\left(\frac{m}{2}\right) + 1$. Using the master theorem results $S(m) = O(m^{\log_2 2})$. Substituting $m = \log n$ gives $T(n) = O(\log n)$.

Problem-43 Find the complexity of the below function.

```
int Function (int n) {  
    if(n <= 2) return 1;  
    else return (Function (floor(sqrt(n))) + 1);  
}
```

Solution: Consider the comments in the function below:

```
int Function (int n) {  
    if(n <= 2) return 1;           //constant time  
    else                           // executes  $\sqrt{n} + 1$  times  
        return (Function (floor(sqrt(n))) + 1);  
}
```

For the above code, the recurrence function can be given as: $T(n) = T(\sqrt{n}) + 1$. This is same as that of [Problem-41](#).

Problem-44 Analyze the running time of the following recursive pseudo-code as a function of n .

```
void function(int n) {  
    if( n < 2 ) return;  
    else counter = 0;  
    for i = 1 to 8 do  
        function ( $\frac{n}{2}$ );  
    for i = 1 to  $n^3$  do  
        counter = counter + 1;  
}
```

Solution: Consider the comments in below pseudo-code and call running time of function(n) as $T(n)$.